System Architecture & Working Documentation

VECTOR8_CPU Project

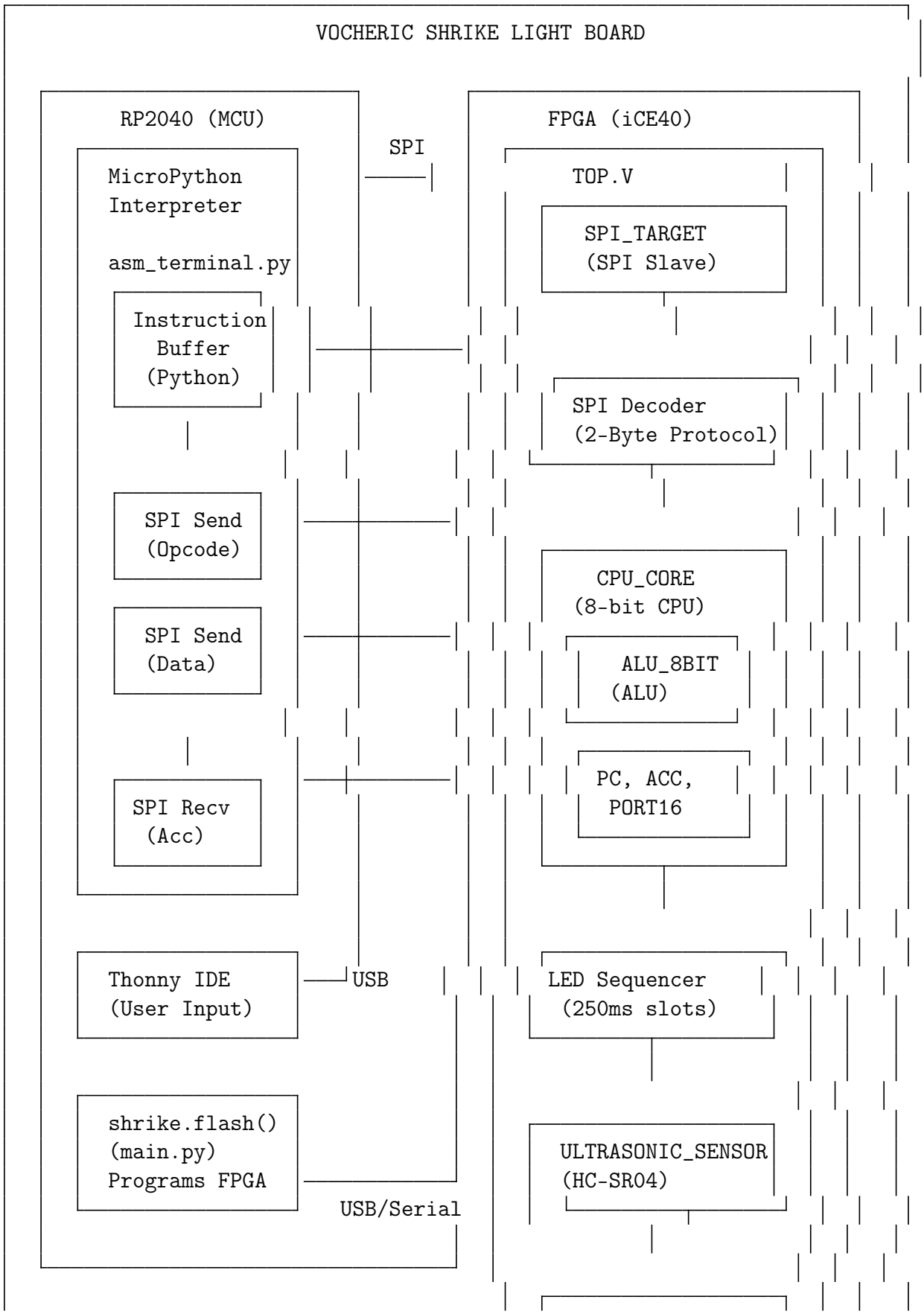
Vocheric Shrike Light FPGA + RP2040

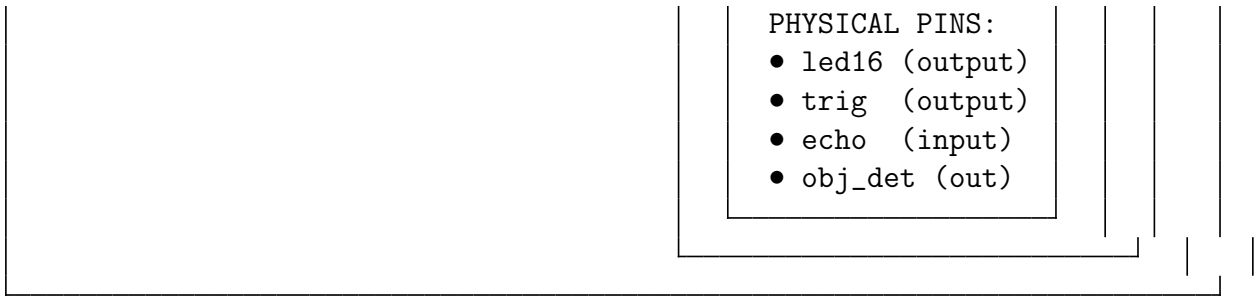
Firmware Development Team

July 16, 2026

Contents

1 Complete System Block Diagram





2 Part 1: FPGA Design Components (What Gets Created Inside the FPGA)

When you run `main.py`, the `FPGA_bitstream_MCU.bin` file configures the iCE40 FPGA to create the following hardware components:

2.1 Component 1: SPI_TARGET (SPI Slave Interface)

```
COMPONENT 1: SPI_TARGET (SPI Slave Interface)
```

- Type: Synthesized digital logic module
- Function: Receives bytes from RP2040 via SPI bus
- FPGA LUTs: ~50-80 LUTs (Look-Up Tables)
- Flip-Flops: ~20-30 FFs (for shift registers and synchronization)
- Created by: spi_target.v
- Connections:
  - Inputs: spi_ss_n, spi_sck, spi_mosi (from RP2040 GPIO pins)
  - Outputs: spi_miso, spi_miso_en (to RP2040 GPIO pins)
  - Internal: o_rx_data[7:0] -> SPI Decoder, i_tx_data <- cpu_acc

How it works:
1. RP2040 sends 8-bit bytes over SPI (Mode 0: CPOL=0, CPHA=0)
2. spi_target shifts in bits on rising edge of SCK
3. After 8 bits, o_rx_data_valid pulses HIGH
4. Simultaneously, cpu_acc value is shifted out on MISO
5. This creates a full-duplex SPI transaction

2.2 Component 2: SPI Decoder (Command Parser)

```
COMPONENT 2: SPI DECODER (Command Parser)
```

- Type: State machine + registers
- Function: Converts SPI bytes into CPU instructions
- FPGA LUTs: ~15-25 LUTs
- Flip-Flops: ~20 FFs (r_opcode[4:0], r_data[7:0], byte_cnt, step_pulse)

- Created by: top.v (lines: "SPI COMMAND DECODER" section)

Protocol: 2-Byte Instruction Format

BYTE 1	BYTE 2
[7:5]=000 [4:0]=op	[7:0]=data

Example: LDA 10
  Byte 1: 0b0000_0001 = 0x01 (opcode for LDA)
  Byte 2: 0b0000_1010 = 0x0A (value 10)

State Machine:
  State 0 (byte_cnt=0): Wait for opcode byte -> store in r_opcode
  State 1 (byte_cnt=1): Wait for data byte -> store in r_data
  After State 1: Generate step_pulse -> CPU executes instruction

2.3 Component 3: CPU_CORE (8-Bit Custom Processor)

COMPONENT 3: CPU_CORE (8-Bit Custom Processor)

- Type: Harvard-style 8-bit CPU (program data comes from SPI, not mem)
- Function: Executes assembly instructions one at a time
- FPGA LUTs: ~80-120 LUTs
- Flip-Flops: ~30-40 FFs (pc[7:0], acc[7:0], z_flag, port16_pattern[7:0])
- Created by: cpu_core.v

Architecture:

REGISTERS:
- PC (Program Counter): 8-bit, 0-255 range
- ACC (Accumulator): 8-bit, main working register
- Z_FLAG: 1-bit, zero flag from last ALU operation
- PORT16_PATTERN: 8-bit, LED output pattern

EXECUTION MODEL:
- NOT a stored-program computer (no program memory)
- Instructions come from RP2040 via SPI in real-time
- Each instruction = 1 clock cycle (single-cycle CPU)
- i_step pulse triggers execution

INSTRUCTION SET (5-bit opcode):

Op	Mnemonic	Description

0x01	LDA	Load immediate
0x02	ADD	Add to accumulator
0x03	SUB	Subtract from accumulator
0x04	AND	Bitwise AND
0x05	OR	Bitwise OR
0x06	XOR	Bitwise XOR
0x07	LSL	Logical Shift Left
0x08	LSR	Logical Shift Right
0x09	ROL	Rotate Left
0x0A	ROR	Rotate Right
0x0B	INC	Increment
0x0C	DEC	Decrement
0x0D	JMP	Jump unconditional
0x0E	JZ	Jump if Zero
0x0F	JNZ	Jump if Not Zero
0x10	SENSE	Read ultrasonic sensor
0x12	OUT	Output to LED port

2.4 Component 4: ALU_8BIT (Arithmetic Logic Unit)

```
COMPONENT 4: ALU_8BIT (Arithmetic Logic Unit)
```

- Type: Combinational logic (no clock, instant output)
- Function: Performs arithmetic and logical operations
- FPGA LUTs: ~30-50 LUTs
- Flip-Flops: 0 FFs (pure combinational)
- Created by: alu_8bit.v

```
Operations:
  ADD: Uses FPGA carry chain for 8-bit addition
  SUB: Uses 2's complement subtraction (a - b = a + ~b + 1)
  AND/OR/XOR: Bitwise operations using LUTs
  Shifts: Wiring operations (no logic needed)
  ROL/ROR: Bit rotation using wiring

Zero Flag:
  Generated by 8-input NOR gate: zero = ~(out[0]|out[1]|...|out[7])
```

2.5 Component 5: ULTRASONIC_SENSOR (HC-SR04 Interface)

```
COMPONENT 5: ULTRASONIC_SENSOR (HC-SR04 Interface)
```

- Type: Timer/counter state machine
- Function: Interfaces with HC-SR04 ultrasonic distance sensor
- FPGA LUTs: ~40-60 LUTs
- Flip-Flops: ~100-150 FFs (32-bit counters)
- Created by: ultrasonic_sensor.v

Sensor Operation:
1. FPGA sends 10us HIGH pulse on trig pin every 60ms
2. Sensor sends back echo pulse (width = time for sound to travel)
3. FPGA measures echo pulse width in clock cycles
4. If width < threshold (20cm max), object_detected = 1
5. Debounce logic prevents flickering

Timing Math (50MHz clock):
- 1 clock cycle = 20 nanoseconds
- 10us trigger pulse = 500 clock cycles
- 60ms period = 3,000,000 clock cycles
- 20cm max distance = ~58,300 clock cycles of echo

2.6 Component 6: LED Sequencer (Port 16 Pattern Display)

COMPONENT 6: LED SEQUENCER (Port 16 Pattern Display)

- Type: Counter + multiplexer
- Function: Displays 8-bit pattern on single LED by time-multiplexing
- FPGA LUTs: ~20-30 LUTs
- Flip-Flops: ~35 FFs (32-bit counter + 3-bit index)
- Created by: top.v ("Port16 pattern sequencer" section)

How it works:
- SLOT_CYCLES = 12,500,000 (250ms at 50MHz)
- bit_index cycles 0-7 every 250ms
- led16 = port16_pattern[7 - bit_index]
- This shows each bit for 250ms, creating a rotating display
- If pattern = 0xAA (10101010), LED blinks ON/OFF every 250ms

3 Part 2: FPGA Resource Utilization Summary

Estimated iCE40 FPGA Resource Usage:

Component	LUTs (est)	FFs (est)	Notes
SPI_TARGET	50–80	20–30	3-stage sync, shift reg
SPI Decoder	15–25	20	State machine
CPU_CORE	80–120	30–40	PC, ACC, flags
ALU_8BIT	30–50	0	Combinational
ULTRASONIC_SENSOR	40–60	100–150	32-bit counters
LED Sequencer	20–30	35	Counter + mux
Top-level routing	20–30	0	Wires, assigns
TOTAL (estimated)	255–395	205–275	

Table 1: FPGA Resource Estimates

iCE40UP5K (used in Shrike Light) has:

- 5,280 LUTs → Your design uses 5-8% of available LUTs
- 128 Kbit RAM → Not used in this design (no memory)
- 8 DSPs → Not used (no multiplication)
- PLL → Used for clock (50MHz input)

Your design is VERY SMALL and leaves plenty of room for expansion!

4 Part 3: Python Code Workflow (What Happens When You Run main.py)

4.1 STEP 1: RUNNING main.py

```
User runs:
python main.py
in terminal
```

```
LINE 1: import shrike
```

- Python loads the shrike module from site-packages
- shrike is a custom library for Vocheric boards
- It imports:
  - USB serial communication classes
  - FPGA programming protocols
  - Board detection and initialization
- The library auto-detects the RP2040 via USB

```
LINE 2: shrike.flash("FPGA_bitstream_MCU.bin")
```

- shrike.flash() function is called
- Parameter: path to compiled FPGA bitstream file

INTERNAL STEPS (performed by shrike library):

```
1. Open USB serial port to RP2040
   - Auto-detects COM port (Windows) or /dev/tty*
   - Sets baud rate (typically 115200 or 921600)

2. Send FPGA programming command to RP2040
   - RP2040 has firmware that understands SPI flash
   - Command puts FPGA into configuration mode

3. Read FPGA_bitstream_MCU.bin from disk
   - File size: typically 50-100 KB for iCE40
   - Binary format: raw bitstream for iCE40 config

4. Stream binary data to RP2040 via USB
   - Data sent in chunks with ACK protocol
   - RP2040 buffers data in its RAM

5. RP2040 programs FPGA configuration memory
   - RP2040 drives SPI pins connected to FPGA
   - FPGA has internal SPI configuration interface
   - Bitstream loaded into FPGA SRAM configuration

6. FPGA reboots with new configuration
   - FPGA reads configuration and sets up LUTs/FFs
   - All your Verilog modules become REAL hardware
   - Physical pins now have the functions you defined
```

RESULT: FPGA is now running your custom CPU design!

5 Part 4: Assembly Terminal Workflow (RP2040 Side)

5.1 WHY DOES THE ASSEMBLER TERMINAL OPEN?

The `asm_terminal.py` script (mentioned in `instructions.txt` but not uploaded) is a MicroPython program that runs ON THE RP2040 itself. Here's the full flow:

```
STEP 1: UPLOAD asm_terminal.py TO RP2040
```

- Connect Shrike Light board to PC via USB
- Open Thonny IDE (Python IDE for MicroPython)
- Copy asm_terminal.py to RP2040's flash storage
- The script is now stored in RP2040's internal flash memory

STEP 2: RUN asm_terminal.py IN THONNY

- Thonny sends command to RP2040 to execute the script
- RP2040's MicroPython interpreter starts running asm_terminal.py
- The script opens a REPL (Read-Eval-Print Loop) terminal
- You see "VECTOR8_CPU ASSEMBLY TERMINAL" prompt

STEP 3: TYPE ASSEMBLY INSTRUCTIONS

- User types: "LDA 10" and presses Enter
- asm_terminal.py parses the line:
  - "LDA" -> opcode lookup table -> 0x01
  - "10" -> decimal parser -> 0x0A
- Stores in buffer: [(0x01, 0x0A)]
- User types "ADD 20" -> stores [(0x01, 0x0A), (0x02, 0x14)]

STEP 4: TYPE 'execute' OR 'loop'

- 'execute' command triggers:

  FOR EACH instruction in buffer:
    1. RP2040 sets spi_ss_n = LOW (select FPGA)
    2. RP2040 sends opcode byte via SPI MOSI
    3. RP2040 toggles SCK 8 times (shift out bits)
    4. FPGA spi_target receives byte, o_rx_data_valid pulses
    5. SPI decoder stores r_opcode = received byte[4:0]
    6. RP2040 sends data byte via SPI MOSI
    7. FPGA receives data, SPI decoder stores r_data
    8. SPI decoder sets step_pulse = 1
    9. CPU_CORE executes instruction in ONE clock cycle
   10. RP2040 reads MISO to get cpu_acc value (optional)
   11. RP2040 sets spi_ss_n = HIGH (deselect FPGA)

- 'loop' command does the same but repeats indefinitely

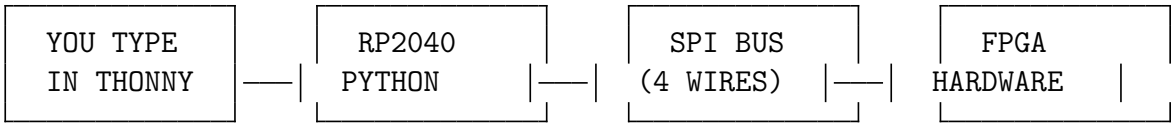
```
    - After last instruction, jumps back to first
    - Creates continuous execution (e.g., sensor monitoring)
```

6 Part 5: Complete Data Flow — Assembly to Hardware Execution

Let's trace what happens when you type this in the terminal:

```
>>> LDA 10
>>> ADD 20
>>> OUTA
>>> execute
```

6.1 FLOW DIAGRAM



6.2 STEP-BY-STEP TRACE

6.2.1 Instruction 1: LDA 10

```
RP2040 sends: Byte1=0x01, Byte2=0x0A

FPGA receives via SPI_TARGET:
  • Byte1 (0x01): r_opcode = 5'b00001
  • Byte2 (0x0A): r_data   = 8'b00001010
  • step_pulse = 1

CPU_CORE executes:
  • pc = pc + 1  (now pc = 1)
  • case(opcode=0x01): acc = alu_out
  • ALU: out = data_in = 10
  • acc = 10, z_flag = 0

RESULT: acc = 10 (0x0A)
```

6.2.2 Instruction 2: ADD 20

```
RP2040 sends: Byte1=0x02, Byte2=0x14

FPGA receives:
  • r_opcode = 5'b00010 (ADD)
  • r_data   = 8'b00010100 (20)
  • step_pulse = 1
```

CPU_CORE executes:
- pc = pc + 1 (now pc = 2)
- case(opcode=0x02): acc = alu_out
- ALU: out = a_reg + data_in = 10 + 20 = 30
- acc = 30, z_flag = 0

RESULT: acc = 30 (0x1E)

6.2.3 Instruction 3: OUT (with accumulator value)

RP2040 sends: Byte1=0x12, Byte2=0x1E (30)

FPGA receives:
- r_opcode = 5'b10010 (OUT)
- r_data   = 8'b00011110 (30)
- step_pulse = 1

CPU_CORE executes:
- pc = pc + 1 (now pc = 3)
- case(opcode=0x12): port16_pattern = data_in
- port16_pattern = 30 (0x1E = 00011110)

LED SEQUENCER:
- bit_index cycles 0-7 every 250ms
- led16 = port16_pattern[7-bit_index]
- Pattern 00011110 displayed bit-by-bit
- LED shows: OFF, OFF, OFF, ON, ON, ON, ON, OFF (repeating)

RESULT: LED displays binary pattern 00011110

7 Part 6: Why Run on RP2040? — Architecture Explanation

QUESTION: Why does the assembly terminal run on RP2040 and not directly on FPGA?

ANSWER: This is a HYBRID architecture with clear role separation:

RP2040 (Microcontroller) ROLE
• Runs MicroPython (high-level language, easy to program)
• Provides USB connectivity to PC (Thonny IDE)
• Runs asm_terminal.py (interactive terminal, user interface)
• Parses human-readable assembly into binary opcodes
• Manages instruction buffer (stores multiple instructions)
• Handles 'loop', 'clear', 'list' commands (control flow)
• Implements DELAY instruction (software timing)

```
• Communicates with FPGA via SPI (hardware interface)
• Can be reprogrammed easily via USB (no FPGA recompilation needed)

ADVANTAGES:
 Easy to modify terminal behavior (just edit Python)
 USB connectivity for development
 Can add new features without changing FPGA design
 Faster development cycle (seconds vs minutes for FPGA synthesis)
```

```
                    FPGA (iCE40) ROLE

• Runs custom VECTOR8_CPU (your Verilog design)
• Executes instructions in REAL HARDWARE (not emulated)
• Interfaces with physical sensors (HC-SR04 ultrasonic)
• Drives physical outputs (LEDs)
• Operates at 50 MHz (much faster than RP2040 Python)
• Deterministic timing (single-cycle execution)
• Parallel processing (ALU, sensor, LED all run simultaneously)

ADVANTAGES:
 Real hardware execution (not simulation)
 Deterministic timing (no Python garbage collection delays)
 Parallel operation (multiple things happen at once)
 Direct hardware control (no OS overhead)
 Persistent operation (runs independently once programmed)
```

7.1 Why Not Run Everything on FPGA?

- **Complexity:** Adding USB, terminal parsing, and file I/O to FPGA would require thousands of LUTs, complex state machines, and external memory. RP2040 already has USB hardware built-in.
- **Development Speed:** Changing Python code: seconds (just save and run); Changing FPGA: minutes (synthesis, place & route, bitstream generation).
- **Flexibility:** Can change terminal commands without FPGA recompilation; add debugging features in Python; save/load programs from RP2040 flash.
- **Cost:** RP2040 + small FPGA is cheaper than large FPGA with everything; RP2040 handles "soft" tasks, FPGA handles "hard" real-time tasks.
- **Educational Value:** Clear separation between software (Python) and hardware (Verilog); students see how they interact; easy to understand where each part runs.

8 Part 7: Physical Connections & Pin Mapping

Shrike Light Board Pin Connections:

RP2040 GPIO	**FPGA PIN**	**FUNCTION**
GPIO0 (MISO)	spi_miso	Data FPGA → RP2040
GPIO1 (CS)	spi_ss_n	Slave Select (active LOW)
GPIO2 (SCK)	spi_sck	SPI Clock
GPIO3 (MOSI)	spi_mosi	Data RP2040 → FPGA
FPGA PIN	**EXTERNAL**	**FUNCTION**
echo	HC-SR04 Echo	Ultrasonic echo input
trig	HC-SR04 Trigger	Ultrasonic trigger output
led16	LED (via resistor)	Pattern display output
object_detected_pin	LED (via resistor)	Direct sensor status LED
clk	50MHz Oscillator	System clock
rst_n	Reset Button	Active-LOW reset

Table 2: Physical Pin Mapping

9 Part 8: Complete Working Summary

```
COMPLETE SYSTEM WORKFLOW

1. DEVELOPMENT PHASE
   ├── Write Verilog code (top.v, cpu_core.v, alu_8bit.v, etc.)
   ├── Synthesize with Yosys (converts Verilog to FPGA netlist)
   ├── Place & Route with nextpnr (maps to physical FPGA resources)
   └── Generate FPGA_bitstream_MCU.bin (final configuration file)

2. FPGA PROGRAMMING PHASE
   ├── Connect Shrike Light board to PC via USB
   ├── Run: python main.py
   ├── shrike library detects RP2040 and opens USB connection
   ├── shrike.flash() sends bitstream to RP2040
   ├── RP2040 programs FPGA via SPI configuration interface
   └── FPGA reboots with your custom CPU design

3. ASSEMBLY EXECUTION PHASE
   ├── Open Thonny IDE
   ├── Upload asm_terminal.py to RP2040
   ├── Run asm_terminal.py (opens interactive terminal)
   ├── Type assembly instructions (LDA, ADD, OUT, SENSE, etc.)
   ├── Type 'execute' or 'loop'
   ├── RP2040 parses assembly -> binary opcodes
   ├── RP2040 sends opcodes+data to FPGA via SPI
   ├── FPGA CPU_CORE executes each instruction in hardware
```

```
      ├── ALU performs arithmetic/logic operations
      ├── Ultrasonic sensor reads distance
      └── LED displays output patterns

  4. RESULT
      ├── You have a CUSTOM 8-bit CPU running inside the FPGA!
      ├── CPU executes YOUR assembly programs in real hardware
      ├── Sensor input affects program execution
      └── LED output shows computation results
```

10 Part 9: File Relationships & Dependencies

FILE DEPENDENCY TREE:

```
FPGA_bitstream_MCU.bin  (generated by synthesis toolchain)
    │
    ├── top.v  (top-level module)
    │       ├── spi_target.v  (SPI slave)
    │       ├── cpu_core.v    (CPU)
    │       │       └── alu_8bit.v  (ALU)
    │       └── ultrasonic_sensor.v  (Sensor)
    │
    └── Generated by: Yosys + nextpnr + icepack

main.py  (Python flash tool)
    └── shrike library  (pre-installed on system)
        └── USB driver  (system-level)

asm_terminal.py  (assembly terminal - mentioned but not uploaded)
    └── Runs on RP2040 MicroPython
        └── Uses machine.SPI  (RP2040 hardware SPI)
            └── Connected to FPGA spi_target
```

11 Part 10: Key Concepts Summary

Concept	Explanation
FPGA Bitstream	Binary file that configures FPGA logic cells
LUT (Look-Up Table)	FPGA building block, implements any logic function
Flip-Flop	FPGA memory element, stores 1 bit
SPI	Serial Peripheral Interface, 4-wire bus protocol
SPI Mode 0	CPOL=0, CPHA=0: clock idle LOW, sample rising edge
Harvard Architecture	Separate program and data paths (here: SPI vs ALU)
Single-Cycle CPU	Each instruction executes in 1 clock cycle
Accumulator	Primary CPU register for arithmetic operations
Opcode	Operation code, tells CPU what to do
Program Counter	Register holding address of current instruction
Zero Flag	Status bit set when ALU result is zero
Synthesis	Converting HDL (Verilog) to gate-level netlist
Place & Route	Mapping netlist to physical FPGA resources
MicroPython	Python 3 subset for microcontrollers
RP2040	Raspberry Pi Pico microcontroller (dual-core ARM)
iCE40	Lattice FPGA family (low-power, small)
HC-SR04	Ultrasonic distance sensor module
Time-Multiplexing	Showing multiple bits on one LED by cycling
Debouncing	Filtering noisy digital signals for stability

Table 3: Key Concepts